§3.2　队列

一、队列的基本概念

1. 定义

- **队列（Queue）**：只允许队尾插入，队头删除的单链表。

  所以队列是阉割版的单链表。

- **特点**：先进先出（FIFO，First In First Out）

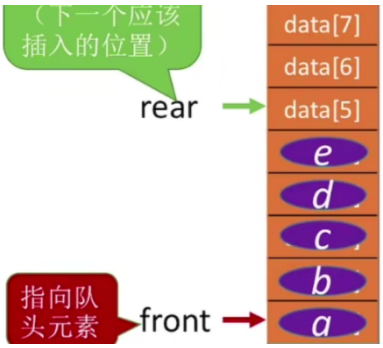
3. 基本操作

操作	功能
`InitQueue(&Q)`	初始化队列
`DestroyQueue(&Q)`	销毁队列，释放内存
`EnQueue(&Q, x)`	入队
`DeQueue(&Q, &x)`	出队
`GetHead(Q, &x)`	读取队头元素，不删除
`QueueEmpty(Q)`	判空

二、队列的顺序实现（循环队列）

1. 结构定义

```
#define MaxSize 10
typedef struct {
    ElemType data[MaxSize];
    int front, rear;
} SqQueue;
```



2. 初始化

```
void InitQueue(SqQueue &Q) {
    Q.front = Q.rear = 0;
}
```

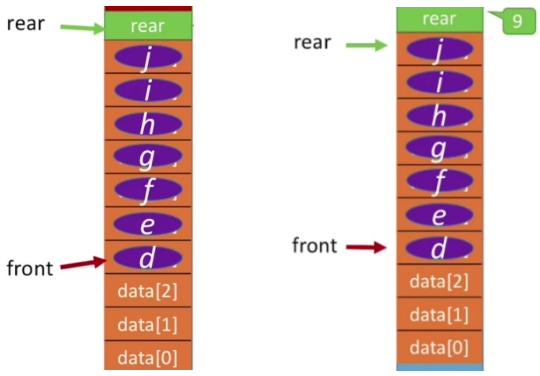
3. 判空（头尾指针相同）

```
bool QueueEmpty(SqQueue Q) {
    return Q.front == Q.rear;
}
```

4. 入队（循环队列）

一般思路：

```
bool EnQueue(SqQueue &Q,ElemType x){
    if(队列已满)
        return false;    //队满则报错
    Q.data[Q.rear]=x;    //将x插入队尾
    Q.rear=Q.rear+1;     //队尾指针后移
    return true;
}
```



如果队头出队了若干元素，此时 rear=MaxSize 也不代表队列已满。所以判断队满的条件是 **if ((Q.rear + 1) % MaxSize == Q.front)**；在代码中 **Q.rear = (Q.rear + 1) % MaxSize** 使得队列成环，rear=9 时下一步就变成 0。

```
bool EnQueue(SqQueue &Q, ElemType x) {
    if ((Q.rear + 1) % MaxSize == Q.front)        // 队满
        return false;
    Q.data[Q.rear] = x;
    Q.rear = (Q.rear + 1) % MaxSize;
    return true;
}
```

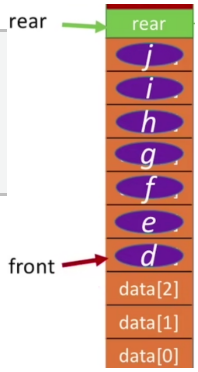
5. 出队（删除一个队头元素，用 x 返回）

```
bool DeQueue(SqQueue &Q, ElemType &x) {
    if (Q.front == Q.rear)          // 队空
        return false;
    x = Q.data[Q.front];
    Q.front = (Q.front + 1) % MaxSize;   //保证 front 转圈遍历
    return true;}
```

注意：把加粗这一行删掉，就是查找（GetHead）代码

6. 队列长度

```
int QueueLength(SqQueue Q) {
    return  (Q.rear + MaxSize - Q.front) % MaxSize;}
```



rear-front 是元素个数绝对值（有可能成环后 rear＜front），

所以加上 MaxSize 再模去相当于取绝对值

ps：(-Q.rear + MaxSize + Q.front) % MaxSize;代表空闲位数

成环后必须要牺牲一个节点空间，即队满时 rear 和 front 差值为 1。若不牺牲空间，front=rear，和之前判空语句一样，引起混淆。

7. 队满判断方案

方案一：牺牲一个存储单元，队满条件为 `(Q.rear + 1) % MaxSize == Q.front`

方案二：增设 `size` 变量

队满：`front == rear && size == MaxSize;`

队空：`front == rear && size == 0;`

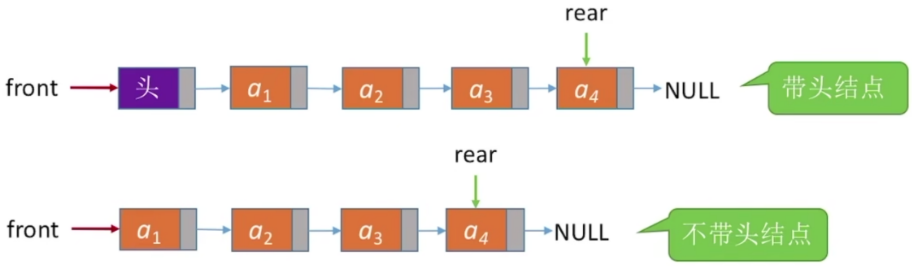
方案二：操作标记符 tag

队满：`front == rear && tag == 1;`

队空：`front == rear && tag == 0;`

```
typedef struct{
ElemType data[MaxSize];
int front,rear;          0  1
int tag;   //最近进行的是删除/插入
} SqQueue;
```

三、队列的链式实现（链队列） 注意：以下代码全部针对带头结点类型



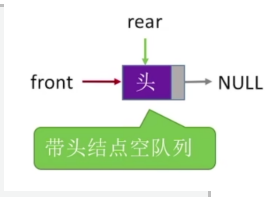
1. 结构定义

```
typedef struct LinkNode {
    ElemType data;
    struct LinkNode *next;
} LinkNode;

typedef struct {
    LinkNode *front, *rear;
} LinkQueue;
```

2. 初始化

```
void InitQueue(LinkQueue &Q) {
    Q.front = Q.rear = (LinkNode*)malloc(sizeof(LinkNode));
    Q.front->next = NULL;
}
```



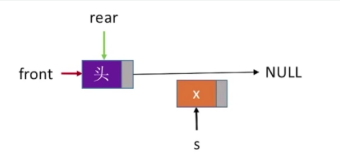
3. 判空

```
bool IsEmpty(LinkQueue Q) {
    return Q.front == Q.rear;      //或者 Q.front->next = NULL
}
```

4.入队（和单链表插入操作一样，遵循“先断后连，先后再前”规则）

```
void EnQueue(LinkQueue &Q, ElemType x) {
    LinkNode *s = (LinkNode*)malloc(sizeof(LinkNode));
```

```
  s->data = x;
    s->next = NULL;
    Q.rear->next = s;
    Q.rear = s;
}
```




```
  s->data = x;
    s->next = NULL;
  if(Q.front == NULL)
  Q.front = s;
  Q.rear = s;
  else{
    Q.rear ->next = s;
    Q.rear = s;
```

5. 出队

```
bool DeQueue(LinkQueue &Q, ElemType &x) {
    if (Q.front == Q.rear) return false;
```

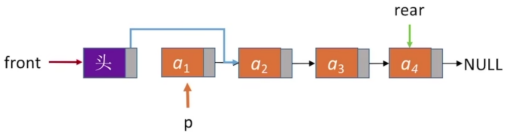
```
  LinkNode *p = Q.front->next;
  x = p->data;
    Q.front->next = p->next;
  //修改 front ->next

    if (Q.rear == p){    //p 是最后一点
  Q.rear = Q.front;}

    free(p);
    return true;}
```

```
   LinkNode *p = Q.front;
  x = p ->data;
  Q.front = p ->next;
  //修改 front

  if(Q.rear == p){    //p 是最后一点
  Q.front = NULL;
  Q.rear = NULL;}
  free(p);
  return true;}
```



左边加横线的，不能写成均指向 NULL。因为均指向头结点，头结点再指向 NULL

对于链式存储，一般不用判满。只有内存不足才会导致队满

四、双端队列

栈：同一端插入、删除

队列：一边插入一遍删除

双端队列：两边插入、两边删除

1. 定义

2. 考点：输出序列合法性

栈：

- 从数学排列角度，总方法数有 n！
- 合法序列数用卡特兰公式$\frac{1}{n+1}C_{2n}^{n}$

- **双端队列：**
- 栈的所有合法序列都合法，部分栈的非法序列，双端队列也可合法输出